

MANUAL DE TECNICO

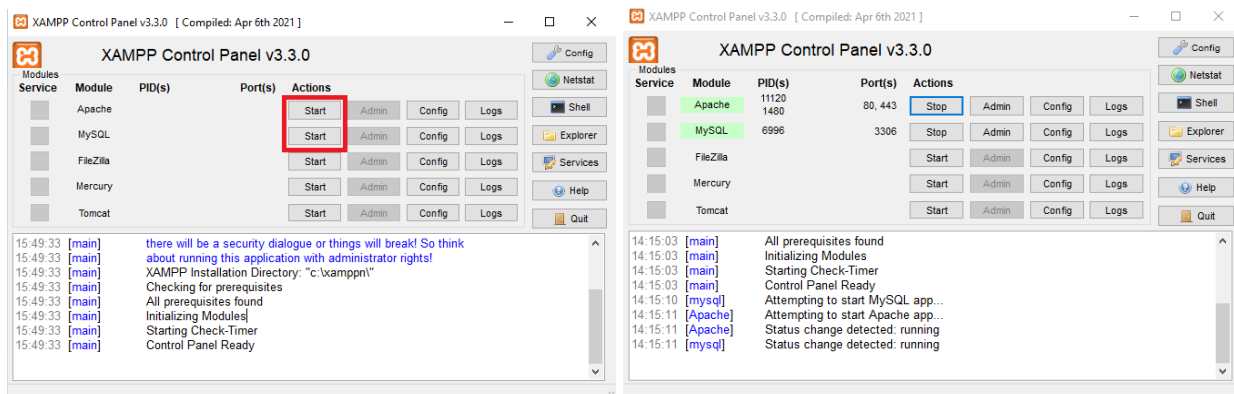
Este manual/guía contiene toda la información y explicaciones necesarias para entender la estructura y el manejo del programa, incluyendo creación de la base de datos, su estructura, su manejo (creacion de tablas, inserción de datos, consultas de datos) desde SQL, su conexión a Python, manejo desde Python, etc.

SQL:

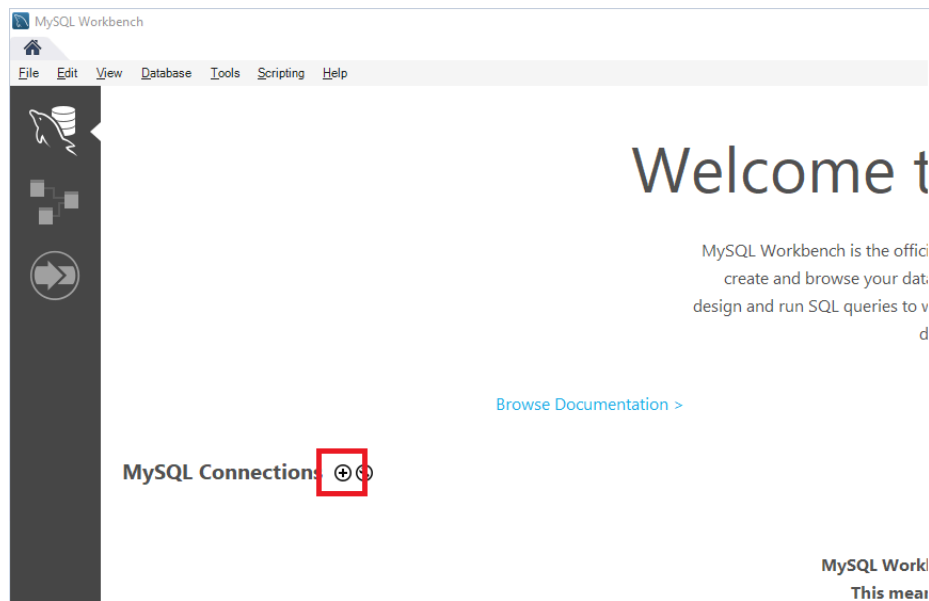
Para poder acceder a la base de datos, deberás acceder a ella a través de MySQL Workbench. Para eso, deberás tener instalado dicho programa, y también XAMPP.

Una vez que tengas ambos programas instalados, descarga el archivo con la base de datos en SQL, a través de [este link](#) de GitHub.

Luego, abre XAMPP y activa los módulos "Apache" y "MySQL" para que MySQL Workbench pueda conectarse a ellos, ya que Workbench es un cliente gráfico que necesita un servidor de base de datos en ejecución para funcionar.

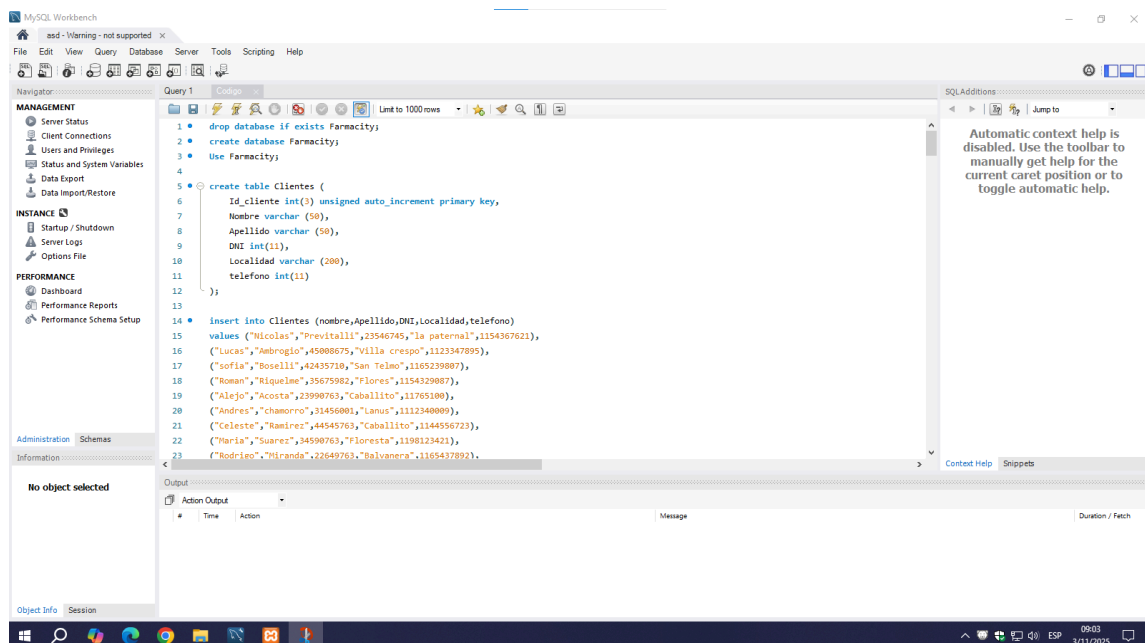


Ahora abre MySQL Workbench y crea una nueva conexión.



Asígnale el nombre que prefieras en “Connection Name”, recomendable que sea un nombre relacionado al proyecto, presione “Ok”. Ahora le aparecerá la conexión lista para abrir. Clickee en ella, le puede salir un cartel que se titule “Connection Warning”, seleccione “Continue Anyway”.

Ahora tendrá una base de datos vacía. Seleccione “File” y luego “Open SQL Script”. Seleccione el archivo SQL que descargó desde GitHub. Le aparecerá todo el código de la base de datos ya hecho.



Para ejecutarlo, clickee sobre el rayito y se mostrarán todas las líneas con un  si no da ningún error, un  si hay una advertencia que no afecte el

funcionamiento del código, o un **✗** si hay un error que pause la compilación (si no hizo ningún cambio al código una vez abierto, debería aparecer todo correcto).



(el rayito con un 1 muestra lo mismo pero línea por línea).

Output				
Action Output				
#	Time	Action	Message	Duration / Fetch
102	09:30:43	insert into Empleados(Id_rol, Nombre, Sueldo, Sector) values(2, "Ana Maria Soldano", 2100.22, 2)	1 row(s) affected	0.000 sec
103	09:30:43	insert into Empleados(Id_rol, Nombre, Sueldo, Sector) values(3, "Marcos Alonso De las Nieves", 1905.12, 3)	1 row(s) affected	0.016 sec
104	09:30:43	insert into Empleados(Id_rol, Nombre, Sueldo, Sector) values(4, "Gonzalo Agustín Celman", 2410.0, 4)	1 row(s) affected	0.000 sec
105	09:30:43	insert into Empleados(Id_rol, Nombre, Sueldo, Sector) values(5, "Juana Sofía Meza", 1420.66, 5)	1 row(s) affected	0.000 sec
106	09:30:43	insert into Empleados(Id_rol, Nombre, Sueldo, Sector) values(6, "Lucio Sepeda", 3000.50, 6)	1 row(s) affected	0.000 sec
107	09:30:43	insert into Empleados(Id_rol, Nombre, Sueldo, Sector) values(7, "Manuela Castillo", 1780.0, 7)	1 row(s) affected	0.000 sec
108	09:30:43	insert into Empleados(Id_rol, Nombre, Sueldo, Sector) values(8, "Manuela Castillo", 2850.0, 8)	1 row(s) affected	0.000 sec
109	09:30:43	insert into Empleados(Id_rol, Nombre, Sueldo, Sector) values(9, "Miguel Velázquez", 2000.3, 9)	1 row(s) affected	0.000 sec
110	09:30:43	insert into Empleados(Id_rol, Nombre, Sueldo, Sector) values(10, "Carmen María Blázquez", 3000.12, 10)	1 row(s) affected	0.000 sec
111	09:30:43	insert into Empleados(Id_rol, Nombre, Sueldo, Sector) values(11, "Denise Ferrandis", 2390.1, 11)	1 row(s) affected	0.000 sec
112	09:30:43	insert into Empleados(Id_rol, Nombre, Sueldo, Sector) values(12, "Ana Isabel Gabasa", 2005.44, 12)	1 row(s) affected	0.000 sec
113	09:30:43	insert into Empleados(Id_rol, Nombre, Sueldo, Sector) values(13, "Juan Francisco Torres", 1600.33, 13)	1 row(s) affected	0.000 sec
114	09:30:43	insert into Empleados(Id_rol, Nombre, Sueldo, Sector) values(14, "Yasmin Alcáide", 2300.33, 14)	1 row(s) affected	0.000 sec
115	09:30:43	insert into Empleados(Id_rol, Nombre, Sueldo, Sector) values(15, "Eric García", 1802.14, 15)	1 row(s) affected	0.000 sec
116	09:30:43	create table Ventas (Id_venta int(3) unsigned auto_increment primary key, Id_medicamentos int(3) unsigned, Id_c...	0 row(s) affected	0.031 sec
117	09:30:43	insert into Ventas(Id_medicamentos, Id_Clientes, Id_Empleados, Fecha_hora, cantidad) values (8,4,4,"2025-9-23 15:30:...	15 row(s) affected Records: 15 Duplicates: 0 Warnings: 0	0.000 sec

PYTHON:

Primero, deberá instalar una versión reciente de python. Una manera fácil de hacerlo es abriendo el cmd (Win + R y escriba "cmd"), y allí escriba "python", eso lo llevará directamente a la tienda de Microsoft donde podrá instalar Python. Una vez instalado, deberá hacer lo mismo con un conector de SQL-Python. En el cmd, pegue este código:

```
python -m pip install mysql-connector-python sqlalchemy pandas python-dotenv
```

y se instalará en unos segundos. Para comprobar que se instaló, escriba `pip list`.

También, deberá instalar un IDE (Entorno de Desarrollo Integrado) que ejecute códigos en Python, como PyCharm o VisualStudio, para poder abrir, manejar y ejecutar muy fácilmente el código.

Una vez que tenga instaladas todas las herramientas, descargue el código .py a través de [este link](#).

Abra su IDE. Cree un nuevo proyecto y abra el código descargado (recuerde que para que éste compile y la conexión sea exitosa, debe

mantener abierto y compilado el código SQL en el MySQL Workbench y dejar activado SQL y Apache en XAMPP).

VisualStudio: seleccione "File", luego "Open file" y seleccione el archivo .py descargado (quizás le salga un cartel, solo apriete "Open" y listo).

Para compilar el código, apriete el  que aparece en la esquina superior derecha. En el monitor serial se imprimirá el menú con el cual el usuario trabaja y hará sus consultas y/o ingresos.

Pycharm: seleccione "Open" y elija el documento descargado. Apriete el  que se encuentra arriba, tirando a la derecha. El código compilará correctamente, abriendo su menú.

Explicaciones de los códigos:

SQL:

La estructura del código consiste en:

Creación de la base de datos

Creación de tabla Ingresos

Creación de tabla Ingresos

Y así...

Tomemos como ejemplo la primer tabla:

- ```
create table Clientes (
 Id_cliente int(3) unsigned auto_increment primary key,
 Nombre varchar (50),
 Apellido varchar (50),
 DNI int(11),
 Localidad varchar (200),
 telefono int(11)
);
```
- ```
insert into Clientes (nombre,Apellido,DNI,Localidad,telefono)
values ("Nicolas","Previtali",23546745,"la paternal",1154367621),
("Lucas","Ambrogio",45008675,"Villa crespo",1123347895),
("sofia","Boselli",42435710,"San Telmo",1165239807),
("Roman","Riquelme",35675982,"Flores",1154329087),
("Alejo","Acosta",23990763,"Caballito",11765100),
("Andres","chamorro",31456001,"Lanus",1112340009),
("Celeste","Ramirez",44545763,"Caballito",1144556723),
("Maria","Suarez",34590763,"Floresta",1198123421),
("Rodrigo","Miranda",22649763,"Balvanera",1165437892),
("Julietta","zanetti",54637201,"Flores",1176893332),
("Luciana","Desimoni",34654987,"Floresta",1145638321),
("Ezequiel","Cerutti",22552225,"Bajo flores",1144339876),
("Damian","Molina",43587322,"Boedo",1144556612),
("Sofia","Acosta",23945763,"Flores",1145632719),
("samuel","fernandez",42531232,"Caballito",1174678901);
```

La primera línea, crea la **tabla** y se le asigna un **nombre**. Dentro de la tabla, se van creando sus **entidades**.

Estas últimas poseen un nombre y un **tipo**. Los tipos de entidad pueden ser "varchar()" si es una cadena de caracteres (entre paréntesis, el límite de caracteres), "int()" o "decimal()" para números enteros o decimales, respectivamente, "datetime()" para fechas y horas, etc.

La entidad a la que se la asigna como "primary key" es aquella que define como único a cada ingreso que se le da a la tabla (un identificador irrepetible). Suele ser el ID y es auto incrementable.

El bloque "insert into" corresponde a los ingresos de información a la tabla (en este proyecto, son 15 ingresos por tabla).

Primero se aclara **a qué tabla** se harán los ingresos (en este caso, "Clientes"), y entre paréntesis se ponen todas las entidades que se usarán en los ingresos (menos el ID, ya que este al ser autoincrementable, se ingresa automáticamente tras el registro).

Posteriormente, se ingresará, también entre paréntesis y separados por comas, la información deseada, (datos como cadenas de texto o fechas, siempre van entre comillas).

Así funciona el resto del código.

En caso de que usted desee hacer modificaciones en el código y testearlo, puede emplear las “Consultas SQL”.

Las consultas SQL permiten obtener, modificar o eliminar información dentro de la base de datos Farmacity.

Por ejemplo, puede realizar una consulta para verificar los medicamentos disponibles junto con su categoría, marca y precio, sin necesidad de hacerlo desde el menú de Python.

Esto resulta útil para testear que las relaciones entre tablas funcionen correctamente o para analizar los datos cargados.

Ejemplo básico de consulta:

SELECT

Medicamentos.Nombre AS Medicamento,

Categorias.descripcion AS Categoria,

Marcas.nombre AS Marca,

Medicamentos.Precio

FROM Medicamentos

JOIN Categorias ON Medicamentos.Id_Cate = Categorias.Id_Categoria

JOIN Marcas ON Medicamentos.id_marc = Marcas.Id_marca;

Esta instrucción mostrará una lista con los nombres de los medicamentos, su categoría, marca y precio actual registrados en la base de datos.

Python:

El código en Python conecta una base de datos MySQL llamada Farmacity, correspondiente al proyecto. Permite realizar distintas operaciones relacionadas con medicamentos, ventas, empleados y clientes. A continuación, se describe el funcionamiento general de cada

bloque de código y las posibles modificaciones que usted le podría implementar.

En primer lugar, el bloque de importaciones.

```
C: > Users > User > Downloads > Codigo_Final.py > ConectarBase
1  import mysql.connector
2  import json
3  from mysql.connector import errorcode
4  import time
5  cursor = None
6  cnx = None
7
```

Este incluye las librerías necesarias: `mysql.connector` para conectar con la base de datos, `json` para generar archivos con formato JSON, `errorcode` para manejar errores específicos y `time` para añadir pausas entre mensajes del menú. Estas librerías son esenciales para la interacción con MySQL y para la creación de archivos. Un técnico podría agregar otras librerías si desea mejorar la interfaz o incluir funciones estadísticas (por ejemplo, `pandas` o `matplotlib`).

La variable “`cnx`” es la conexión activa con la base de datos MySQL, usada para enviar y confirmar las operaciones, y “`cursor`” ejecuta las consultas SQL dentro de esa conexión y devuelve los resultados, permitiendo insertar, leer o modificar datos.

La función `ConectarBase()`, como su nombre lo indica, intenta establecer una conexión con MySQL utilizando las credenciales definidas (usuario, contraseña, host y nombre de base de datos). Si la conexión falla, muestra distintos mensajes de error según el tipo de problema.

```
def ConectarBase():
    global cnx, cursor
    try:
        cnx = mysql.connector.connect(user="root", password="", host="localhost", database="Farmacity")
        cursor = cnx.cursor(dictionary=True)
        print('Conexión establecida')

    except mysql.connector.Error as err:
        if err.errno == errorcode.ER_ACCESS_DENIED_ERROR:
            print('Usuario o contraseña incorrectos!')
        elif err.errno == errorcode.ER_BAD_DB_ERROR:
            print('La base de datos no existe!')
        else:
            print(err)

def ConsultaInsertar(meds,clients,employes,datexd, cantidad):
    sql = "INSERT INTO Ventas (Id_medicamentos,Id_Clientes,Id_Empleados,Fecha_yhora,cantidad)VALUES( %s, %s, %s, %s,%s)"
    cursor.execute(sql,(meds,clients,employes,datexd,cantidad))
    cnx.commit()
    return cursor.lastrowid

ConectarBase()
```

La función `ConsultaInsertar()` permite registrar una nueva venta en la tabla Ventas. Recibe los ID's del medicamento, el cliente, empleado, fecha y cantidad vendida. Ejecuta la consulta SQL correspondiente y guarda los cambios con `commit()`. Retorna el ID de la venta recién insertada.

```
def consultaInsertar(meds,clients,empleyes,datexd, cuantity): 1 usage
# meds: El id del medicamento
#clients: El id del cliente
#empleyes: El id del empleado
#datexd: La fecha de la compra con la hora
#cuantity: La cantidad del producto comprado
#sql: El insert a la base de datos
sql = "INSERT INTO Ventas (Id_medicamentos,Id_Clientes,Id_Empleados,Fecha_yhora,cantidad)VALUES( %s, %s, %s, %s,%s)"
cursor.execute(sql,(meds,clients,empleyes,datexd,cuantity))
cnx.commit()
return cursor.lastrowid
```

La función `aplicar_contra_indicaciones()` contiene una lista con advertencias específicas de distintos medicamentos y otra lista paralela con los IDs correspondientes de cada medicamento. Son textos informativos que el programa muestra según el ID del medicamento ingresado. Usted podría actualizar o ampliar estas descripciones, agregar nuevas o eliminar alguna en caso de que se descontinúe ese o esos medicamentos.

```
def aplicar_contra_indicaciones():
#lista: donde se van a almacenar los diccionarios
#contraindicaciones-donde se guardan las contraindicaciones
#ids:donde se guardan los ids
#medicamento: diccionario base para crear los demas
lista=[]
contraindicaciones=["personas alérgicas a la amoxicilina, a otras penicilinas o a antibióticos betalactámicos","no debe utilizarse en personas con alergia a la sustancia o sus componentes, en m
"personas con antecedentes de alergias graves a este medicamento, especialmente anafilaxia, enfermedad del suero u otras reacciones alérgicas previas.", "hipersensibilidad al
"hipersensibilidad a las benzodiazepinas, insuficiencia respiratoria grave, apnea del sueño, miastenia gravis y en ciertas condiciones del embarazo y lactancia.",
"sangrado activo, úlceras pépticas, insuficiencia hepática o renal grave, hipersensibilidad al acenocumarol, embarazo y ciertas condiciones médicas que aumentan el riesgo de
"con alergia al metazolol, antecedentes de agranulocitosis, asma, problemas de médula ósea, porfiria hepática aguda, insuficiencia renal o hepática grave, embarazo en el terc
]
ids=[3,4,6,7,11,12,15]
for i in range(len(ids)):
    medicamento = {
        "id": ids[i],
        "contraindicacion": contraindicaciones[i]
    }
    lista.append(medicamento)
return lista
```

Opciones del menú:

La función `dar_datos()` recopila la información de una nueva venta, llama a la función `aplicar_contra_indicaciones()` para que, en caso de que el medicamento vendido sea uno de los que contiene una advertencia, muestre esta misma y luego llama a `ConsultaInsertar()` para registrar la venta.

La función Medicamentos() realiza una consulta que devuelve todos los medicamentos registrados en la base. Se usa principalmente en otros módulos del programa.

La función Binario() simula una búsqueda binaria, dividiendo la lista en dos mitades y busca por posición del índice. Permite identificar el medicamento correspondiente a un ID ingresado.

El bloque proximos_a_vencer() obtiene los tres medicamentos más próximos a su fecha de caducidad mediante una consulta SQL ordenada por fecha ascendente. Esto permite detectar productos que deben ser retirados o promocionados. La función crear_Archivo_Proximos_Vencer() usa esa información para generar un archivo JSON con el nombre "Consulta1.json", el cual contiene los datos obtenidos. Usted puede modificarlo para que, por ejemplo, cree un archivo .txt si lo desea.

El bloque ventas() realiza una consulta que cuenta la cantidad de ventas por empleado, agrupando los registros según el identificador del empleado. La función crear_Archivo_ventas() exporta estos datos en un archivo JSON llamado "Consulta2.json", que permite tener un resumen de productividad por trabajador.

Las funciones sortear_meds() y mostrar_sortear() permiten listar los medicamentos pertenecientes a una categoría específica. El usuario ingresa el ID de la categoría, y el programa muestra los productos correspondientes junto con su precio y stock. Estas funciones pueden modificarse para mostrar también la marca o el proveedor. Similarmente, las funciones Cod_barras() y mostrar_barras() permiten buscar medicamentos a partir de su código de barras. Esto sería útil para un sistema con lector físico de códigos, ya que en este menú, los códigos deben ingresarse a mano.

Finalmente, el bloque Menu() organiza todas las funciones dentro de un menú interactivo. Presenta al usuario las distintas opciones disponibles: registrar ventas, buscar productos, crear archivos de reportes, filtrar medicamentos por categoría o código de barras, y salir del sistema. Usa

pausas cortas con `time.sleep()` para que vayan apareciendo de a poco y así ser más cómodo visualmente.

Codigo del menu:

```

159 def menu():
160     #counter: Es lo que permite que se repita indefinidamente, cambia de valor al terminarse de ejecutar
161     #el programa
162     #opcion: La seleccion del cajero para ver que es lo que requiere hacer
163     counter=True
164     while counter==True:
165         time.sleep(0.25)
166         print("////////////////////////////////////")
167         time.sleep(0.25)
168         print("// 1. Para registrar ventas  ////")
169         time.sleep(0.25)
170         print("// 2. Busqueda de producto  //")
171         time.sleep(0.25)
172         print("/// 3. Imprimir productos a vencer ///")
173         time.sleep(0.25)
174         print("// 4. Imprimir ventas por empleados /")
175         time.sleep(0.25)
176         print("// 5. Seleccionar Meds por Categoria/")
177         time.sleep(0.25)
178         print("// 6. Seleccionar Meds por Barras  /")
179         time.sleep(0.25)
180         print("// Otro para salir  //")
181         time.sleep(0.25)
182         print("////////////////////////////////////")
183         time.sleep(0.25)
184         opcion=int (input(""))
185         if opcion==1:
186             dar_datos()
187         elif opcion==2:
188             binario()
189         elif opcion==3:
190             crear_Archivo_Proximos_Vencer()
191             print("Se ha creado el archivo correctamente")
192         elif opcion==4:
193             crear_Archivo_ventas()
194             print("Se ha creado el archivo correctamente")
195         elif opcion==5:
196             mostrar_sortear()
197         elif opcion==6:
198             mostrar_barras()
199         else:
200             if cnx.is_connected():
201                 cnx.close()
202                 print ("Adios que tenga un buen dia")
203                 counter=False
204
205
206 menu()
```

Impresión:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

PS C:\Users\User\AppData\Local\Programs\Microsoft Windows Common-
Conexión establecida
////////////////////////////////////
// 1. Para registrar ventas  ////
// 2. Busqueda de producto  //
/// 3. Imprimir productos a vencer ///
// 4. Imprimir ventas por empleados /
// 5. Seleccionar Meds por Categoria/
// 6. Seleccionar Meds por Barras  /
// Otro para salir  //
////////////////////////////////////

```